

PlaybookIQ

Product Requirement Document (PRD) — Enterprise Sports Intelligence Platform

Target Client:	Sportsnexa	Author:	AI Systems Engineer
Core Tech Stack:	AWS Bedrock, OpenSearch, Streamlit	Design System:	IBM Carbon (Gray 10 Light)

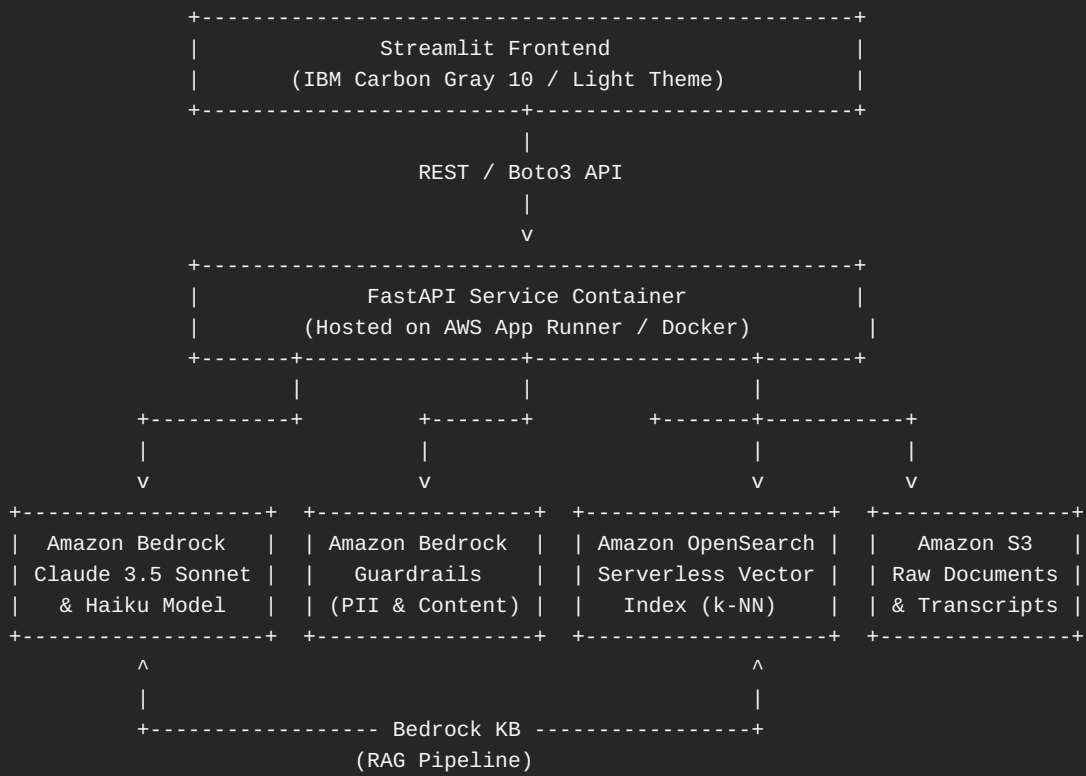
1. Executive Summary & Architecture Overview

1.1 Project Objectives

PlaybookIQ is an enterprise-grade AI scouting and analytics platform designed for sports organizations (franchises, media, and betting analytics). The platform ingests unstructured data—game transcripts, player scouting reports, injury logs, and tactical playbooks—and provides semantic retrieval, automated tactical summaries, and interactive multi-agent reasoning.

This project serves as a production-level demo demonstrating core competencies in **Amazon Bedrock**, **Claude 3.5 Sonnet / Haiku**, **Amazon OpenSearch Serverless**, **RAG architectures**, **Bedrock Agents / MCP**, and containerized deployment on **AWS App Runner**.

1.2 System Architecture Diagram



2. Tech Stack & Local Environment Setup

2.1 Technology Matrix

Component	Technology Selection	Role / Specification
Primary Language	Python 3.11+	Core development runtime
Package Manager	uv	Fast, isolated dependency management
LLM Engine	Amazon Bedrock	Claude 3.5 Sonnet & Claude 3 Haiku
Embedding Model	Amazon Titan Text Embeddings V2	1024-dimension vector generation
Vector Store	Amazon OpenSearch Serverless	Vector Search Collection (hnsw k-NN)
Storage	Amazon S3	Raw sports transcripts & scouting PDFs
API Framework	FastAPI	High-performance microservice REST backend
UI Framework	Streamlit	IBM Carbon Gray 10 Light Design System
IaC & Deployment	AWS CDK / Docker / App Runner	Infrastructure as Code & Container Hosting

2.2 Local Workspace Initialization (uv Workflow)

Execute the following commands in VS Code terminal:

```
# Initialize uv project workspace
uv init playbook-iq
cd playbook-iq

# Add core dependencies
uv add streamlit fastapi uvicorn boto3 opensearch-py langchain-community pydantic python-dotenv

# Activate virtual environment
source .venv/bin/activate # On Linux/macOS
```

Project Directory Structure:

```

playbook-iq/
├── .env
├── pyproject.toml
├── Dockerfile
├── app/
│   ├── __init__.py
│   ├── main.py                # FastAPI entry point
│   ├── ui.py                  # Streamlit frontend (IBM Carbon design)
│   └── services/
│       ├── __init__.py
│       ├── bedrock_service.py # Bedrock & Guardrails wrappers
│       ├── rag_service.py     # OpenSearch k-NN & Knowledge Base integration
│       └── agent_service.py   # Bedrock Agents & Action Groups / MCP
├── templates/
│   └── carbon_theme.css      # Carbon Light styling overrides
├── data/
│   ├── sample_scouting_report.pdf
│   └── sample_game_transcript.txt
└── cdk/
    └── app.py                # AWS CDK infrastructure definition

```

3. Data Ingestion & AWS Bedrock RAG Pipeline

3.1 Vector Index Schema (Amazon OpenSearch Serverless)

Create an OpenSearch Serverless Vector Collection with dimension **1024** matching Titan Embeddings V2.

```

{
  "settings": {
    "index.knn": true
  },
  "mappings": {
    "properties": {
      "vector_field": {
        "type": "knn_vector",
        "dimension": 1024,
        "method": {
          "name": "hnsw",
          "engine": "nmslib",
          "space_type": "cosinesimil"
        }
      },
      "text": { "type": "text" },
      "document_type": { "type": "keyword" },
      "player_id": { "type": "keyword" },
      "timestamp": { "type": "date" }
    }
  }
}

```

3.2 Ingestion Engine & Retrieval Wrapper (rag_service.py)

```
import boto3
import json
from typing import List, Dict, Any

class BedrockRAGService:
    def __init__(
        self,
        region_name: str = "us-east-1",
        kb_id: str = "YOUR_BEDROCK_KB_ID"
    ):
        self.bedrock_agent_runtime = boto3.client(
            "bedrock-agent-runtime", region_name=region_name
        )
        self.kb_id = kb_id

    def query_knowledge_base(
        self,
        query: str,
        number_of_results: int = 5
    ) -> List[Dict[str, Any]]:
        response = self.bedrock_agent_runtime.retrieve(
            knowledgeBaseId=self.kb_id,
            retrievalQuery={"text": query},
            retrievalConfiguration={
                "vectorSearchConfiguration": {
                    "numberOfResults": number_of_results
                }
            }
        )

        results = []
        for match in response.get("retrievalResults", []):
            results.append({
                "content": match["content"]["text"],
                "score": match.get("score", 0.0),
                "location": match.get("location", {})
            })
        return results
```

4. Bedrock Model Integration & Guardrails (bedrock_service.py)

```
import boto3
import json

class BedrockLLMService:
    def __init__(
        self,
        region_name: str = "us-east-1",
        guardrail_id: str = None,
        guardrail_version: str = "DRAFT"
    ):
        self.bedrock = boto3.client("bedrock-runtime", region_name=region_name)
        self.sonnet_model_id = "anthropic.claude-3-5-sonnet-20240620-v1:0"
        self.haiku_model_id = "anthropic.claude-3-haiku-20240307-v1:0"
        self.guardrail_id = guardrail_id
        self.guardrail_version = guardrail_version

    def generate_tactical_summary(
        self,
        prompt: str,
        context: str,
        use_fast_model: bool = False
    ) -> str:
        model_id = self.haiku_model_id if use_fast_model else self.sonnet_model_id

        system_prompt = (
            "You are PlaybookIQ, an elite enterprise sports intelligence analyst. "
            "Provide precise, highly tactical game insights based solely on retrieved contexts. "
            "If context is insufficient, state that clearly."
        )

        user_content = f"Context Data:\n{context}\n\nUser Query:\n{prompt}"

        payload = {
            "anthropic_version": "bedrock-2023-05-31",
            "max_tokens": 1500,
            "system": system_prompt,
            "messages": [
                {"role": "user", "content": user_content}
            ],
            "temperature": 0.2
        }

        kwargs = {
            "modelId": model_id,
            "contentType": "application/json",
            "accept": "application/json",
            "body": json.dumps(payload)
        }

        if self.guardrail_id:
            kwargs["guardrailIdentifier"] = self.guardrail_id
            kwargs["guardrailVersion"] = self.guardrail_version

        response = self.bedrock.invoke_model(**kwargs)
        response_body = json.loads(response.get("body").read())

        return response_body["content"][0]["text"]
```

5. Agent & Tool Orchestration (MCP / Action Groups)

Bedrock Agents execute dynamic tool calling for real-time external API interactions (e.g., retrieving live box scores or player health telemetry).

Action Group Definition (get_player_stats tool)

```
{
  "name": "GetPlayerStats",
  "description": "Fetch real-time seasonal metrics and physical telemetry for a specific player.",
  "parameters": {
    "player_name": {
      "type": "string",
      "description": "Full name of the athlete (e.g. Patrick Mahomes)",
      "required": true
    },
    "season": {
      "type": "integer",
      "description": "Four-digit year (e.g. 2025)",
      "required": false
    }
  }
}
```

6. Frontend: IBM Carbon Design System (Light Theme)

The interface follows the IBM Carbon **Gray 10** Light aesthetic: high clarity, strict grid structure, flat 1px neutral borders, monospaced data visualization, and zero high-contrast gradients.

6.1 Custom Carbon Styling Overlay (templates/carbon_theme.css)

```
/* IBM Carbon Gray 10 Light Theme Overrides */
:root {
  --cds-background: #f4f4f4;
  --cds-layer-01: #ffffff;
  --cds-layer-02: #f4f4f4;
  --cds-text-01: #161616;
  --cds-text-02: #525252;
  --cds-border-subtle: #e0e0e0;
  --cds-border-strong: #8d8d8d;
  --cds-interactive: #0f62fe;
}

.stApp {
  background-color: var(--cds-background);
  color: var(--cds-text-01);
  font-family: 'IBM Plex Sans', -apple-system, BlinkMacSystemFont, sans-serif;
}

div[data-testid="stSidebar"] {
  background-color: var(--cds-layer-01);
  border-right: 1px solid var(--cds-border-subtle);
}

.carbon-card {
  background-color: var(--cds-layer-01);
  border: 1px solid var(--cds-border-subtle);
  padding: 16px;
  margin-bottom: 12px;
  border-radius: 0px;
}

.carbon-header {
  font-size: 1.25rem;
  font-weight: 600;
  letter-spacing: 0px;
  color: var(--cds-text-01);
  border-bottom: 1px solid var(--cds-border-subtle);
  padding-bottom: 8px;
  margin-bottom: 16px;
}

.stButton>button {
  background-color: var(--cds-interactive);
  color: #ffffff;
  border: none;
  border-radius: 0px;
  font-size: 14px;
  padding: 8px 16px;
}

.stButton>button:hover {
  background-color: #0353e9;
  color: #ffffff;
}
```

6.2 Streamlit Application Implementation (app/ui.py)


```

import streamlit as st
from services.bedrock_service import BedrockLLMService
from services.rag_service import BedrockRAGService

st.set_page_config(
    page_title="PlaybookIQ | Sports Intelligence Platform",
    page_icon="⚡",
    layout="wide"
)

# Inject Carbon Light CSS
with open("app/templates/carbon_theme.css") as f:
    st.markdown(f"<style>{f.read()}</style>", unsafe_allow_html=True)

# Header Section
st.markdown('''
<div class="carbon-card">
    <div style="display: flex; justify-content: space-between; align-items: center;">
        <div>
            <h2 style="margin: 0; font-weight: 600;">PlaybookIQ Analytics Platform</h2>
            <span style="color: #525252; font-size: 13px;">AWS Bedrock • OpenSearch Serverless •
Claude 3.5 Sonnet</span>
        </div>
        <div>
            <span style="background: #e0e0e0; color: #161616; padding: 4px 8px; font-family:
monospace; font-size: 12px;">ENV: AWS-US-EAST-1</span>
        </div>
    </div>
</div>
''', unsafe_allow_html=True)

# Main Grid Layout
col_sidebar, col_main = st.columns([1, 3])

with col_sidebar:
    st.markdown('<div class="carbon-header">Configuration</div>', unsafe_allow_html=True)
    model_choice = st.selectbox("Model Endpoint", ["Claude 3.5 Sonnet", "Claude 3 Haiku"])
    rag_enabled = st.checkbox("Enable RAG Context (OpenSearch)", value=True)
    guardrails_active = st.checkbox("Enforce Bedrock Guardrails", value=True)

    st.markdown('<div class="carbon-header" style="margin-top:24px;">Filters</div>',
unsafe_allow_html=True)
    sport = st.selectbox("Sport Category", ["NFL Football", "NBA Basketball", "Global Soccer"])
    confidence_threshold = st.slider("Min Similarity Score", 0.0, 1.0, 0.75)

with col_main:
    st.markdown('<div class="carbon-header">Query Workspace</div>', unsafe_allow_html=True)
    user_query = st.text_area("Input Tactical Query or Scouting Request", height=100,
        placeholder="e.g., Extract third-down blitz pass coverage
vulnerabilities from the past 3 games.")

    if st.button("Execute Intelligence Query"):
        if user_query:
            with st.spinner("Executing hybrid retrieval and Claude inference..."):
                # Simulated RAG Retrieval
                rag_service = BedrockRAGService()
                llm_service = BedrockLLMService(
                    guardrail_id="g-sample123" if guardrails_active else None
                )

                contexts = []
                if rag_enabled:
                    raw_docs = rag_service.query_knowledge_base(user_query)

```

```

        contexts = [d["content"] for d in raw_docs if d["score"] >= confidence_threshold]

        joined_context = "\n---\n".join(contexts) if contexts else "No high-confidence
documents retrieved."

        response_text = llm_service.generate_tactical_summary(
            prompt=user_query,
            context=joined_context,
            use_fast_model=(model_choice == "Claude 3 Haiku")
        )

        # Output Cards
        st.markdown('<div class="carbon-header" style="margin-top:16px;">System Synthesis</
div>', unsafe_allow_html=True)
        st.markdown(f'<div class="carbon-card">{response_text}</div>', unsafe_allow_html=True)

        if rag_enabled and contexts:
            with st.expander("Retrieved Context Chunks (OpenSearch Vector Store)":
                for idx, ctx in enumerate(contexts):
                    st.code(f"// Reference Chunk [{idx+1}]\n{ctx}", language="text")

```

7. Containerization & Deployment (Docker + AWS App Runner)

7.1 Production Dockerfile

```

FROM python:3.11-slim

# Install uv package manager
COPY --from=ghcr.io/astral-sh/uv:latest /uv /bin/uv

WORKDIR /app

# Copy dependency specifications
COPY pyproject.toml uv.lock ./

# Install project dependencies
RUN uv sync --frozen --no-cache

# Copy application files
COPY app/ ./app/

EXPOSE 8501

ENV PYTHONUNBUFFERED=1

CMD ["uv", "run", "streamlit", "run", "app/ui.py", "--server.port=8501", "--server.address=0.0.0.0"]

```

7.2 AWS CDK Stack (cdk/app.py)

```
from aws_cdk import (
    Stack,
    aws_s3 as s3,
    aws_opensearchserverless as aoss,
    aws_bedrock as bedrock,
    aws_apprunner_alpha as apprunner,
    App
)
from constructs import Construct

class PlaybookIQStack(Stack):
    def __init__(self, scope: Construct, id: str, **kwargs) -> None:
        super().__init__(scope, id, **kwargs)

        # 1. Document Bucket
        doc_bucket = s3.Bucket(self, "PlaybookDocsBucket")

        # 2. OpenSearch Serverless Collection
        vector_collection = aoss.CfnCollection(
            self, "VectorStore",
            name="playbookiq-vectors",
            type="VECTORSEARCH"
        )

        # 3. Bedrock Knowledge Base setup (Logical reference)
        # Connected via OpenSearch Serverless and Titan Embeddings
```

8. Key Interview Talking Points (AWS Strategic Mastery)

When discussing this implementation with technical interviewers at **Sportsnexa**, emphasize these core architectural decisions:

1. Why Bedrock Knowledge Bases vs. Pinecone / Standalone Vector DBs?

Answer: Native AWS integration ensures zero-egress security compliance, unified IAM role delegation, zero key management overhead, and seamless ingestion directly from S3.

2. How does Bedrock Guardrails protect enterprise solutions?

Answer: Guardrails filter out toxic content, enforce PII redaction (e.g., removing player medical/legal info), and prevent hallucinated out-of-domain queries before requests reach Claude models.

3. Optimizing Latency and Cost with Model Routing:

Answer: Route real-time, low-latency UI auto-completions to **Claude 3 Haiku**, while reserving **Claude 3.5 Sonnet** for deep multi-document synthesis and tactical analysis.

4. Why OpenSearch Serverless?

Answer: Automatic scaling down to zero compute units during off-hours, avoiding fixed cluster costs while providing production-ready k-NN search performance.